

Autonomous Self-Healing Code Agent

Professional architecture, engineering roadmap, safety model, evaluation harness, and implementation plan

Project objective

Build an evidence-driven Site Reliability Engineering agent that detects production incidents, correlates telemetry with the exact deployed source code, reproduces failures in isolation, creates constrained patches, verifies them, and opens reviewable draft pull requests.

The first professional release should stop at automated draft pull requests. Automatic merge and deployment belong to later maturity levels because they require stronger evidence, policy, rollback, and organizational controls.

1. Product principles

This project succeeds only when it reduces incident response time without trading away reliability. The agent must decline a patch when the evidence points to infrastructure, configuration, invalid data, capacity, security policy, or an external dependency.

- Evidence before modification
- Exact deployed revision before repository analysis
- Reproduction before claiming a fix
- Tests before pull-request creation
- Least privilege for every tool and integration
- Immutable audit records for every decision and action
- Explicit abstention when evidence is weak
- Human review before merge during the portfolio and initial production stages

2. Target architecture

End-to-end flow

Microservices

→ OTLP traces, logs, and metrics

OpenTelemetry Collector

→ Trace backend: Jaeger or Tempo

→ Incident detector

Incident detector

→ Normalize, enrich, fingerprint, deduplicate, and score

Incident store + durable workflow engine

→ Repository context engine

→ Model gateway and agent harness

→ MCP tool server

Disposable repair sandbox

→ Reproduce → Patch → Verify → Risk review

GitHub App

→ Draft pull request + check run + evidence

Architectural planes

- Detection: decide whether telemetry represents a real incident that deserves investigation.
- Investigation: correlate the incident with the exact deployed code and collect relevant evidence.
- Repair: reproduce and patch the issue inside a disposable workspace.
- Governance: verify, explain, audit, and publish a controlled pull request.

3. Recommended technology stack

- Core application: TypeScript with strict compiler settings on the current Node.js LTS release.
- Workspace: pnpm workspaces and Turborepo.
- HTTP services: Fastify with Zod schemas.
- Telemetry: OpenTelemetry SDKs and Collector, with Jaeger locally.
- Production-style observability: Grafana Tempo, Loki, Prometheus, and Grafana behind provider interfaces.
- Workflow orchestration: Temporal for durable, retryable incident workflows.
- State and artifacts: PostgreSQL plus object storage for large trace, log, test, and patch artifacts.
- Code intelligence: Tree-sitter, ripgrep, TypeScript Language Service, Git history, and test discovery.
- Tool interface: MCP TypeScript SDK behind an internal policy layer.
- Testing: Vitest, integration tests, contract tests, Testcontainers, and fault-injection fixtures.
- Isolation: Docker for the MVP; gVisor or ephemeral Kubernetes jobs later.
- Source control: GitHub App with Octokit and GitHub Checks.
- Infrastructure: Docker Compose locally and Kubernetes with Helm or Kustomize as the production exercise.

4. Telemetry and incident detection

Do not create an agent run for every error span. A single outage can emit thousands of nearly identical failures. Group and enrich related telemetry before starting an expensive diagnosis.

Required telemetry metadata

- `service.name`
- `service.version` containing the Git commit SHA
- Deployment environment
- Container image digest or deployment identifier
- HTTP route, RPC method, or messaging operation
- Trace ID and span ID
- Exception type, message, and stack trace
- Request correlation identifiers
- Source maps for compiled TypeScript

Incident detector pipeline

- Validate and normalize incoming telemetry.
- Retrieve the full distributed trace.
- Sanitize secrets, personal data, authorization headers, cookies, and payload fields.
- Generate an incident fingerprint.
- Deduplicate occurrences inside a configurable time window.
- Calculate severity, novelty, error rate, affected services, and deployment correlation.
- Start a repair workflow only when repository and environment policies permit it.

Suggested fingerprint

```
service.name + environment + exception.type  
  
+ normalized exception.message  
  
+ top application stack frame + route or operation
```

Normalize UUIDs, timestamps, order identifiers, memory addresses, and other changing values before hashing. Store both the normalized fields and hash so the behavior remains explainable.

Trigger sources

- New exception fingerprint
- Sudden error-rate increase
- SLO burn-rate alert
- Repeated crash loop
- Failure beginning immediately after a deployment
- Cross-service contract or schema error

5. Incident contract and persistent state

Every repair run must be reproducible. Store an immutable incident bundle containing:

- Incident ID, fingerprint, first seen, last seen, and occurrence count
- Service, environment, region, deployed Git SHA, and image digest
- Trace identifiers and selected spans
- Sanitized exception attributes and related logs
- Metric snapshots and deployment events
- Repository and ownership metadata
- Agent, model, prompt, policy, and tool-schema versions
- Every tool request, result, timeout, retry, and failure
- Generated patches, test artifacts, risk assessment, and final outcome

Valid final outcomes

- Code patch proposed
- Configuration problem detected
- Dependency or infrastructure problem detected
- Rollback recommended
- Insufficient evidence
- Reproduction failed
- Patch failed verification
- Draft pull request created

Abstention is required. A system that always produces a patch will eventually create persuasive but incorrect changes.

6. Repository context engine

Tree-sitter supplies the structural index. Lexical search, language-service information, Git history, and test relationships complete the retrieval system.

Index generated for each commit

- Files and detected languages
- Classes, interfaces, functions, methods, parameters, and return types
- Imports, exports, and symbol locations
- Call relationships where reliable
- Associated test files and fixtures
- Configuration, manifests, ownership, and build metadata

Retrieval order

- Exact file and line from the stack trace
- Enclosing function, method, or class
- Callers, callees, imports, and exported symbols
- Other services present in the failed trace
- Tests associated with the suspected symbol
- Recent commits affecting the same code

- Keyword and semantic search as fallback

Cache repository maps by Git SHA. Never analyze the default branch when the running service was built from a different commit. Add embeddings only after evaluation shows a measurable retrieval gap.

7. Agent harness and workflow

The harness owns state, model turns, tool execution, budgets, policy checks, persistence, retries, and termination. Use a typed state machine instead of an open-ended conversational loop.

```
INGEST → TRIAGE → LOCALIZE → FORM_HYPOTHESIS
→ REPRODUCE → PLAN_PATCH → APPLY_PATCH
→ VERIFY → RISK_REVIEW → CREATE_DRAFT_PR
```

Any state may exit as `ABSTAINED`, `FAILED`, `NEEDS_HUMAN`, or `COMPLETED`.

Execution budgets

- Maximum model turns and token usage
- Maximum tool calls and repeated identical actions
- Wall-clock and test timeouts
- Maximum patch size and files modified
- Cost per incident
- Maximum context and tool-output size

Use native structured tool calls. Keep concise hypotheses, evidence, and decisions in state, and store tool activity independently. Do not depend on literal Thought, Action, and Observation text.

Minimal repair state

RepairState

```
incident: IncidentBundle
repository: RepositorySnapshot
evidence: Evidence[]
hypotheses: Hypothesis[]
currentPlan?: RepairPlan
patch?: PatchArtifact
validation: ValidationResult[]
risk: RiskAssessment
budgets: ExecutionBudgets
```

8. Tools and MCP boundary

Expose small typed capabilities instead of a general-purpose shell. The policy engine stays outside the model and decides which tool can run for each repository, environment, and incident.

Read-only telemetry tools

- `getIncident`
- `getTrace`
- `searchRelatedLogs`
- `getServiceDependencies`
- `getDeploymentMetadata`

Read-only repository tools

- `readFileRange`
- `searchCode`
- `getSymbol`
- `getRelatedSymbols`
- `findTests`
- `getGitHistory`
- `getChangedFilesSinceDeployment`

Sandbox mutation tools

- `createWorkspace`
- `applyPatch`
- `writeRegressionTest`
- `formatChangedFiles`
- `runAllowedTask`
- `getDiff`

GitHub tools

- `createBranch`
- `commitPatch`
- `openDraftPullRequest`
- `publishCheckRun`

Tool contract requirements

- Strict JSON Schema
- Canonical path normalization and repository-root containment
- Time, memory, output, and concurrency limits
- Permission classification and policy evaluation
- Structured errors and retry classification
- Audit events and idempotency keys

Create an allowlisted command catalog for install, build, typecheck, lint, focused tests, service tests, and integration tests.

9. Model strategy

Keep providers behind a common interface and benchmark them on the incident suite. Route by task instead of using the most expensive model everywhere.

- Deterministic TypeScript for parsing, deduplication, policy, and simple classification
- A fast inexpensive model for summaries and routing
- A cost-balanced reasoning model for fault localization
- A strong coding model for difficult diagnosis and patch generation
- A fresh verifier that sees the incident, patch, and test evidence without inheriting the patching model's assumptions
- An optional local open-weight model for sensitive classification or redaction

Current benchmark candidates include GPT-5.6 Luna, Terra, and Sol; Claude Sonnet 5 and Opus 5; and Gemini 3.8 Flash. Do not make the architecture depend on one model.

Do not fine-tune initially. Build the evaluation set first and separate failures caused by retrieval, tools, orchestration, prompts, model capability, or missing tests. Version every prompt and model configuration.

10. Repair sandbox and security

Repository content, logs, dependencies, generated patches, and proposed commands are untrusted inputs.

Per-incident sandbox procedure

- Resolve the exact deployed commit.
- Create a disposable writable workspace.
- Run as a non-root user with production credentials removed.
- Mount only the repository and controlled dependency caches.
- Deny network access by default.
- Apply CPU, memory, process, disk, and wall-clock limits.
- Collect the diff, test output, build artifacts, and audit events.
- Destroy the workspace after completion.

Security controls

- Treat source comments, logs, issue text, and test output as prompt-injection material.
- Enforce permissions in executable code outside the model.
- Redact secrets and personal data before remote model requests.
- Use short-lived repository-scoped GitHub App installation tokens.
- Keep branch protection and required checks enabled.
- Never mount deployment credentials into a patch-generation sandbox.

11. Verification ladder

- Reproduce the original failure.
- Show that a focused regression test fails before the patch.
- Show that the same test passes after the patch.

- Run existing related tests.
- Run type checking, linting, and formatting.
- Run the complete affected-service suite.
- Run integration or contract tests.
- Build the changed service.
- Confirm only policy-approved files changed.
- Calculate patch risk and run an independent review.

A test run only after the patch does not prove that it detects the original defect. Store before-and-after results whenever reproduction is possible.

12. GitHub integration and autonomy

Use a GitHub App with minimum repository permissions. Create a dedicated incident branch, commit only the verified diff, publish a check run, and open a draft pull request.

Autonomy ladder

- L0: produce an incident diagnosis.
- L1: generate and verify a local patch.
- L2: open a draft pull request with evidence.
- L3: open a ready-for-review pull request for narrowly approved low-risk changes.
- L4: automatically merge narrowly approved changes.
- L5: deploy, observe, and roll back automatically.

The portfolio target is L2. It already demonstrates observability, agent orchestration, code intelligence, safe execution, verification, and source-control integration.

Pull-request evidence

- Incident fingerprint and trace link
- Detected production version
- Root-cause hypothesis and evidence
- Reproduction command and before/after result
- Patch explanation and affected files
- Tests, checks, and build results
- Risk assessment and known limitations
- Agent, model, prompt, and policy versions
- Complete audit-record link

13. Evaluation harness

Build an incident-specific benchmark before optimizing models. Each fixture contains a repository commit, service definitions, failure trigger, sanitized telemetry bundle, expected faulty files, hidden verification tests, optional reference patch, and fault category.

Required fault families

- Null or undefined input
- Cross-service schema drift
- Incorrect validation

- Retry or timeout mistake
- Race condition
- Resource leak
- Bad feature flag or environment configuration
- Dependency outage
- Database constraint failure
- Authentication or authorization regression
- Regression introduced by a recent commit

Evaluation metrics

- Incident classification accuracy
- Top-1 and top-3 faulty-file localization
- Failure reproduction success
- Patch application success
- Correct patch rate against hidden tests
- Regression and false-positive rates
- Abstention precision
- Median time to diagnosis and verified patch
- Model cost, tokens, and tool calls per successful repair
- Unrelated files changed
- Draft-PR acceptance and merge rate

Use SWE-bench for general repair behavior, but retain telemetry-driven fixtures because generic coding benchmarks do not evaluate trace correlation, grouping, deployed-version resolution, or PR governance.

14. Operational metrics

- Mean time to detect
- Time to a useful diagnosis
- Time to a verified patch
- End-to-end mean time to recovery
- Code versus non-code classification accuracy
- Reproduction, verification, and draft-PR success rates
- Human acceptance, edit, rejection, and merge rates
- Escaped-regression and reopened-incident rates
- Agent cost and resource usage per incident

Measure diagnosis, patch proposal, merge, deployment, and recovery separately.

15. Suggested repository structure

```
self-healing-sre/
├─ apps/
|   └─ order-service/
```

```
| |─ payment-service/
| |─ incident-api/
| |─ repair-worker/
| |─ dashboard/
|─ packages/
| |─ contracts/
| |─ telemetry/
| |─ incident-engine/
| |─ repo-indexer/
| |─ context-engine/
| |─ agent-core/
| |─ model-gateway/
| |─ mcp-tools/
| |─ sandbox-runner/
| |─ policy-engine/
| |─ github-integration/
| |─ observability/
|─ evals/{fixtures,fault-injection,hidden-tests,reports}/
|─ infra/{docker-compose,otel-collector,temporal,grafana,kubernetes}/
|─ docs/{architecture,adr,threat-model,runbooks,demos}/
|─ .github/workflows/
```

16. Sixteen-week implementation roadmap

Weeks 1–2: instrumented playground

- Build OrderService and PaymentService.
- Inject deterministic null, schema, and timeout failures.
- Send traces through the Collector and view failures in Jaeger.

Exit gate: one repeatable request produces a correctly connected failed trace.

Weeks 3–4: source correlation

- Add service version, environment, image digest, exception attributes, and TypeScript source maps.
- Resolve stack frames to the deployed Git revision.

Exit gate: a trace resolves to the correct repository, commit, source file, and line.

Week 5: incident detector

- Implement validation, redaction, fingerprinting, deduplication, severity, and deployment correlation.

Exit gate: repeated errors create one incident with an occurrence count.

Weeks 6–7: repository context

- Create the Tree-sitter symbol map and ranked retrieval.
- Connect symbols to tests and recent Git history.

Exit gate: the faulty symbol appears in the top three retrieved candidates.

Week 8: read-only agent

- Implement the state machine, model gateway, budgets, and evidence store.
- Support code, non-code, and insufficient-evidence outcomes.

Exit gate: the agent produces an auditable diagnosis without modifying a repository.

Weeks 9–10: reproduction and patching

- Build disposable workspaces and the allowlisted test runner.
- Require focused reproduction, a minimal patch, and before/after validation.

Exit gate: known incidents produce minimal patches that pass hidden tests.

Week 11: MCP and policy

- Expose typed tools through MCP and enforce path, command, time, output, and permission boundaries.

Exit gate: all file and command actions are validated and audited.

Week 12: GitHub integration

- Create the GitHub App, incident branches, commits, checks, and draft pull requests.

Exit gate: a passing repair opens a draft PR with complete evidence.

Weeks 13–14: evaluation and model comparison

- Build 30–50 varied fixtures.
- Compare context strategies, prompts, models, latency, cost, and abstention.

Exit gate: evaluation runs are reproducible and results are published.

Week 15: security and recovery

- Test redaction, prompt injection, path traversal, network denial, resource limits, retries, and worker crashes.

Exit gate: security and recovery tests pass.

Week 16: product demonstration

- Complete the dashboard, ADRs, threat model, runbooks, demo script, and recorded walkthrough.

Exit gate: a fresh environment reproduces the complete incident-to-draft-PR story.

17. First end-to-end release scope

- Two TypeScript services in one repository
- One local environment with OpenTelemetry Collector and Jaeger
- Three injected fault classes
- Exact Git SHA and source-map correlation
- Tree-sitter plus lexical search
- One coding model behind a provider-neutral interface
- Docker repair sandbox
- Regression-test generation and validation
- Draft pull-request creation
- Twenty evaluation fixtures
- No automatic merge or deployment

18. Portfolio demonstration

The strongest demonstration is a single command that launches the platform, injects traffic, displays the failed distributed trace, creates one deduplicated incident, resolves the deployed source revision, retrieves the relevant symbols and tests, reproduces the failure, adds a regression test, generates a minimal patch, verifies it, and opens a draft pull request with the complete evidence trail.

The README and demo should report localization accuracy, correct-patch rate, abstention behavior, median diagnosis time, verified-patch time, model cost, and known limitations.

Authoritative references

- [OpenTelemetry Collector](#) — Collector architecture and components.
- [OpenTelemetry exception conventions](#) — Recording exceptions and stack-trace attributes.
- [Tree-sitter parser guide](#) — Parsing source code into syntax trees.
- [Tree-sitter query syntax](#) — Extracting language-specific nodes and symbols.
- [Temporal documentation](#) — Durable workflows, retries, task queues, and recovery.
- [Model Context Protocol specification update](#) — Current protocol architecture and authorization direction.
- [GitHub App permissions](#) — Least-privilege permissions for GitHub Apps.
- [GitHub Checks API](#) — Publishing checks and annotations on pull requests.
- [gVisor documentation](#) — Stronger isolation for untrusted workloads.
- [Kubernetes NetworkPolicy](#) — Pod ingress and egress isolation.
- [SWE-bench evaluation guide](#) — Containerized patch evaluation.
- [OpenAI model catalog](#) — Current OpenAI model options.
- [Anthropic model status](#) — Current Claude model availability.
- [Gemini model catalog](#) — Current Gemini model options.